哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期：2025 春季

课程名称：人工智能（实验）

实验名称：实验二：深度学习

实验性质：综合设计型

实验学时：4 地点：T2612

学生班级：1 班

授课教师：

实验与创新实践教育中心制

2025 年 4 月

说明：

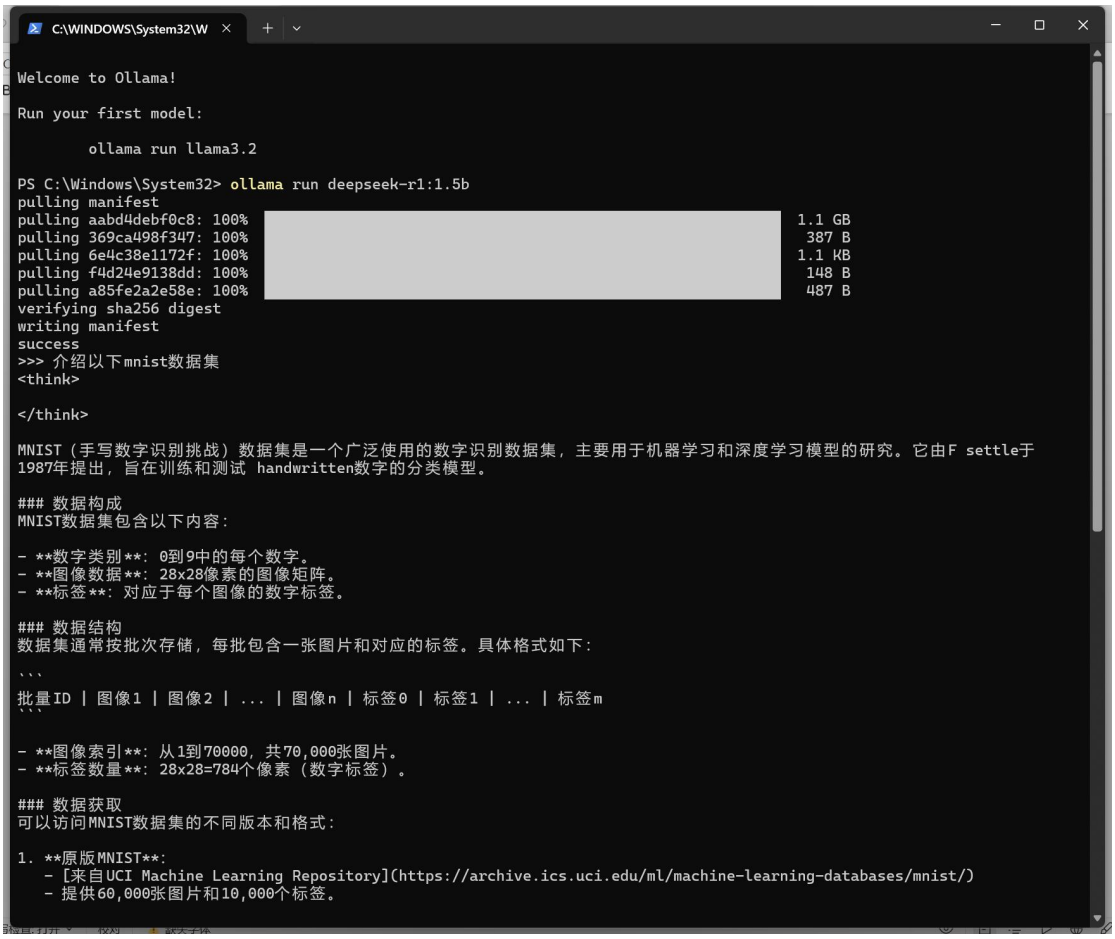
1、有修改的代码源文件需提交，数据集和模型参数不要提交。

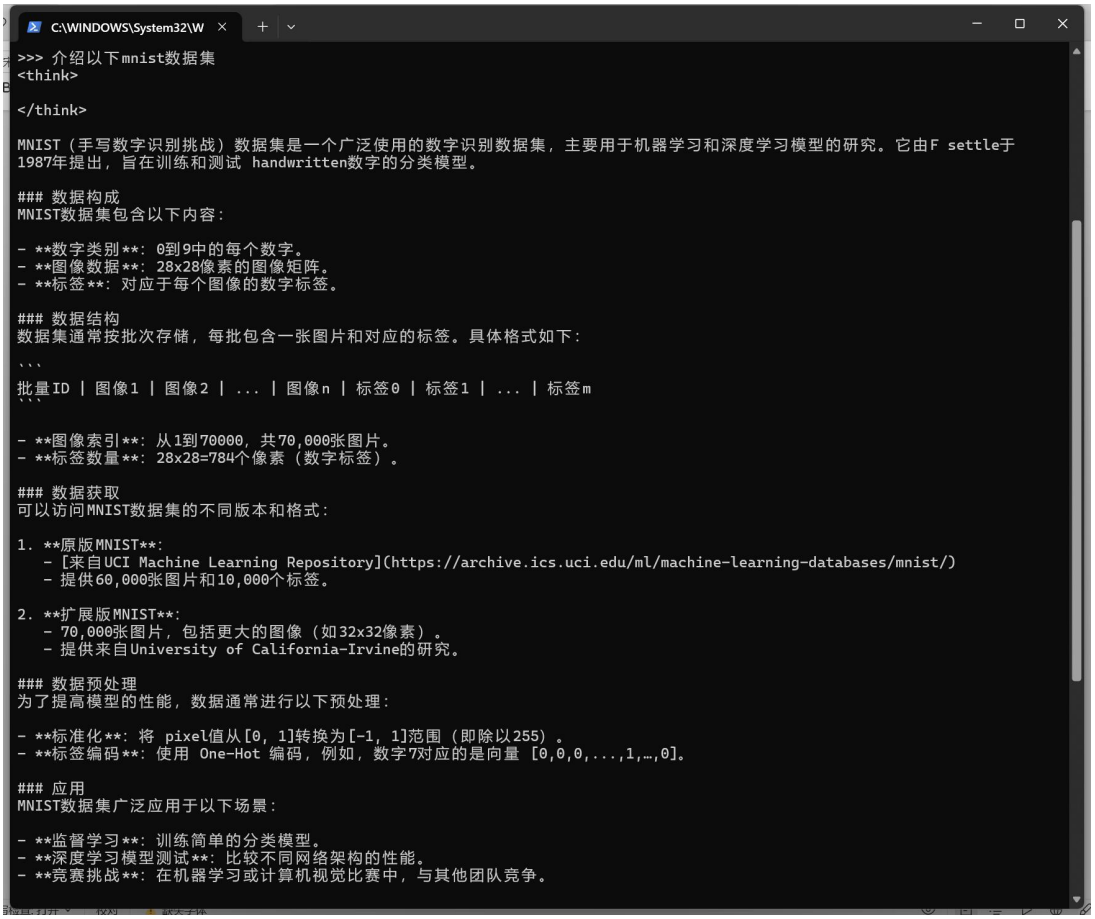
2、可以自行用 markdown 格式最后导出 pdf，排版要一致，封面只需体现对应的信息。

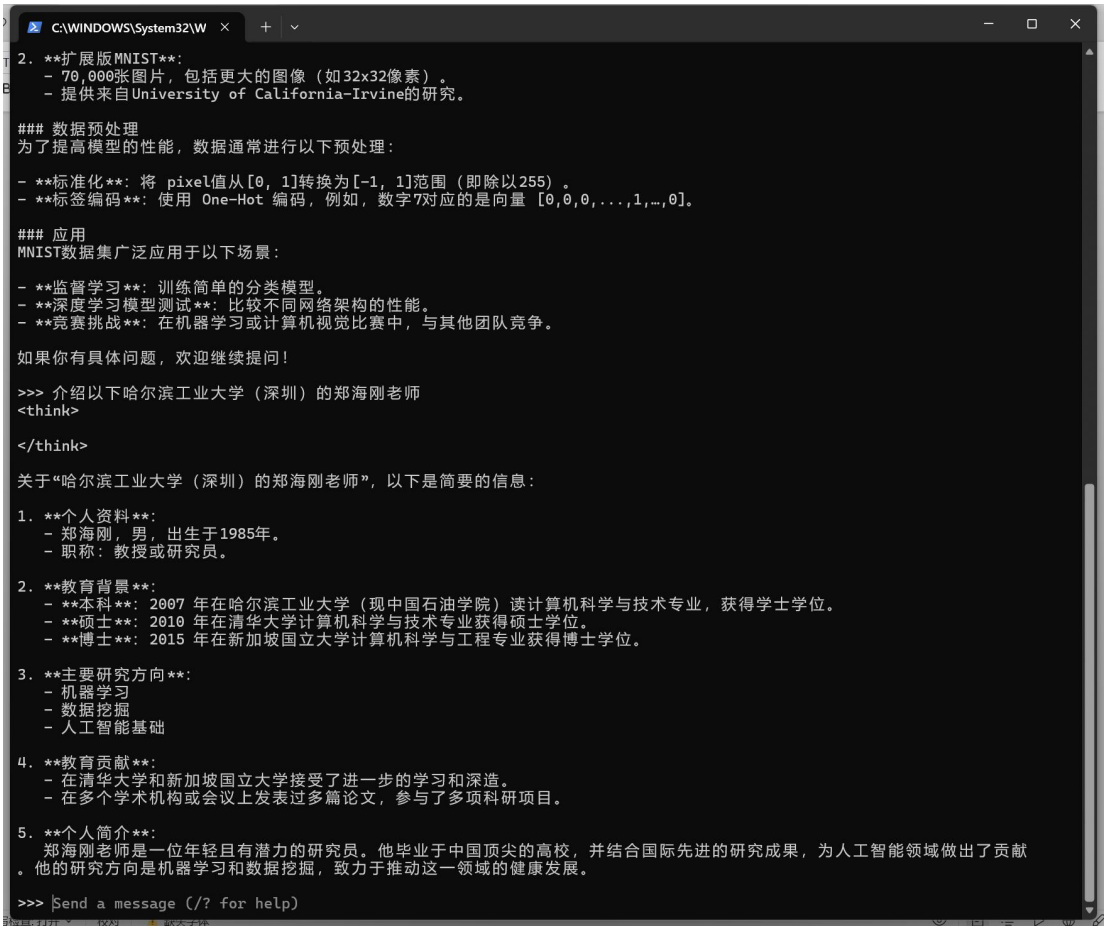
3、训练有准确率的要求，注意看指导书；模板中的说明文字可以删除。

使用 Ollama 本地部署 DeepSeek 蒸馏版

使用 deepseek 1.5b




C:\WINDOWS\System32\W
>>> 介绍以下mnist数据集
<think>
</think>
MNIST（手写数字识别挑战）数据集是一个广泛使用的数字识别数据集，主要用于机器学习和深度学习模型的研究。它由F settle于1987年提出，旨在训练和测试 handwritten数字的分类模型。
数据构成
MNIST数据集包含以下内容：
- **数字类别**：0到9中的每个数字。
- **图像数据**：28x28像素的图像矩阵。
- **标签**：对应于每个图像的数字标签。
数据结构
数据集通常按批次存储，每批包含一张图片和对应的标签。具体格式如下：
```
批量ID | 图像1 | 图像2 | ... | 图像n | 标签0 | 标签1 | ... | 标签m
```
- **图像索引**：从1到70000，共70,000张图片。
- **标签数量**：28x28=784个像素（数字标签）。
数据获取
可以访问MNIST数据集的不同版本和格式：
1. **原版MNIST**：
- [来自UCI Machine Learning Repository](https://archive.ics.uci.edu/ml/machine-learning-databases/mnist/)
- 提供60,000张图片和10,000个标签。
2. **扩展版MNIST**：
- 70,000张图片，包括更大的图像（如32x32像素）。
- 提供来自University of California-Irvine的研究。
数据预处理
为了提高模型的性能，数据通常进行以下预处理：
- **标准化**：将 pixel值从[0, 1]转换为[-1, 1]范围（即除以255）。
- **标签编码**：使用 One-Hot 编码，例如，数字7对应的是向量 [0,0,0,...,1,…,0]。
应用
MNIST数据集广泛应用于以下场景：
- **监督学习**：训练简单的分类模型。
- **深度学习模型测试**：比较不同网络架构的性能。
- **竞赛挑战**：在机器学习或计算机视觉比赛中，与其他团队竞争。


C:\WINDOWS\System32\W
2. **扩展版MNIST**：
- 70,000张图片，包括更大的图像（如32x32像素）。
- 提供来自University of California-Irvine的研究。
数据预处理
为了提高模型的性能，数据通常进行以下预处理：
- **标准化**：将 pixel值从[0, 1]转换为[-1, 1]范围（即除以255）。
- **标签编码**：使用 One-Hot 编码，例如，数字7对应的是向量 [0,0,0,...,1,…,0]。
应用
MNIST数据集广泛应用于以下场景：
- **监督学习**：训练简单的分类模型。
- **深度学习模型测试**：比较不同网络架构的性能。
- **竞赛挑战**：在机器学习或计算机视觉比赛中，与其他团队竞争。
如果你有具体问题，欢迎继续提问！
>>> 介绍以下哈尔滨工业大学（深圳）的郑海刚老师
<think>
</think>
关于“哈尔滨工业大学（深圳）的郑海刚老师”，以下是简要的信息：
1. **个人资料**：
- 郑海刚，男，出生于1985年。
- 职称：教授或研究员。
2. **教育背景**：
- **本科**：2007 年在哈尔滨工业大学（现中国石油学院）读计算机科学与技术专业，获得学士学位。
- **硕士**：2010 年在清华大学计算机科学与技术专业获得硕士学位。
- **博士**：2015 年在新加坡国立大学计算机科学与工程专业获得博士学位。
3. **主要研究方向**：
- 机器学习
- 数据挖掘
- 人工智能基础
4. **教育贡献**：
- 在清华大学和新加坡国立大学接受了进一步的学习和深造。
- 在多个学术机构或会议上发表过多篇论文，参与了多项科研项目。
5. **个人简介**：
郑海刚老师是一位年轻且有潜力的研究员。他毕业于中国顶尖的高校，并结合国际先进的研究成果，为人工智能领域做出了贡献。他的研究方向是机器学习和数据挖掘，致力于推动这一领域的健康发展。
>>> Send a message (/? for help)

```
>>> deepseek是什么
<think>

</think>

DeepSeek是中国国内知名的人工智能搜索引擎，专注于智能信息检索和自然语言理解。它能够通过算法分析和推理，处理复杂的文本
内容，并提供即时的响应。

### 主要功能
1. **知识库检索**:
   - 通过数据库搜索文档或文章，获取相关的信息。

2. **自然语言理解与生成**:
   - 分析中文、英文等语言输入，理解其含义并生成相应的回应。
   - 支持的领域包括翻译、问答、对话等。

3. **智能交互**:
   - 用于在线聊天、问答系统和对话框中提供帮助。

4. **多模态处理**:
   - 处理文本、图像、音频等多种数据类型，提升识别度。

### 发展历程
DeepSeek的开发主要基于深度学习算法，如卷积神经网络（CNN）、循环网络（RNN）等。它的核心是通过大量的人工智能训练和优化
，提高其性能和适应性。

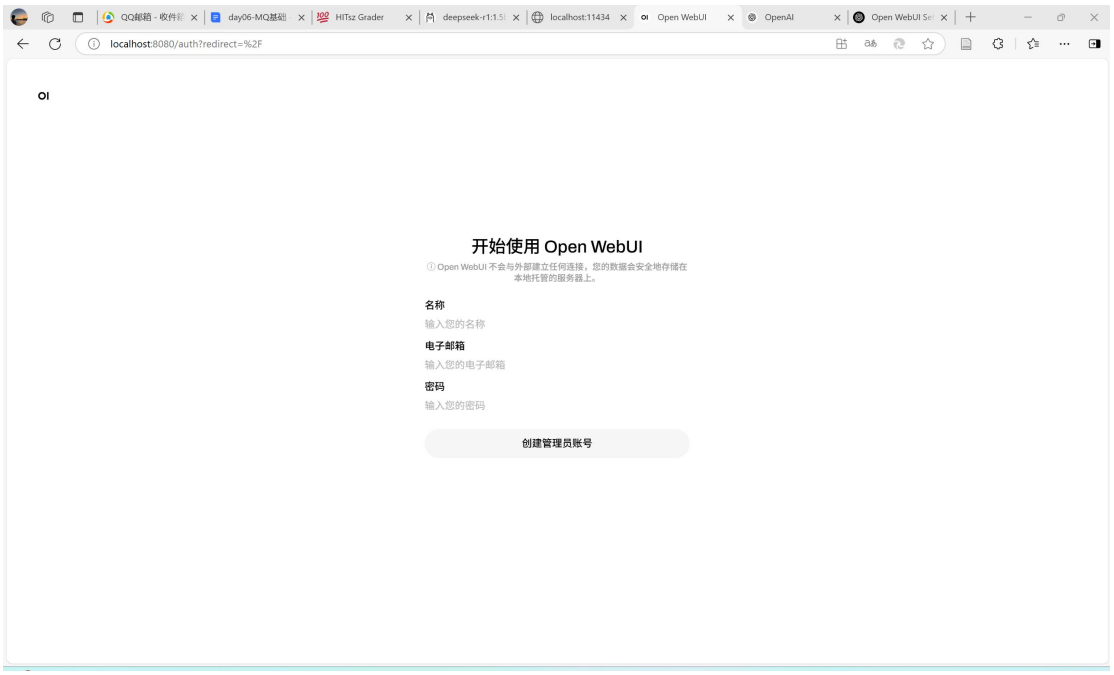
如果你对DeepSeek感兴趣，可以通过以下途径获取更多信息：

- **官方网站**：访问中国科学院人工智能研究所的官网。
- **学术论文**：查找与DeepSeek相关的研究成果。

希望这对你有帮助！

>>> Send a message (/? for help)
```

使用 Open WebUI 支持浏览器访问





1、Mindspore

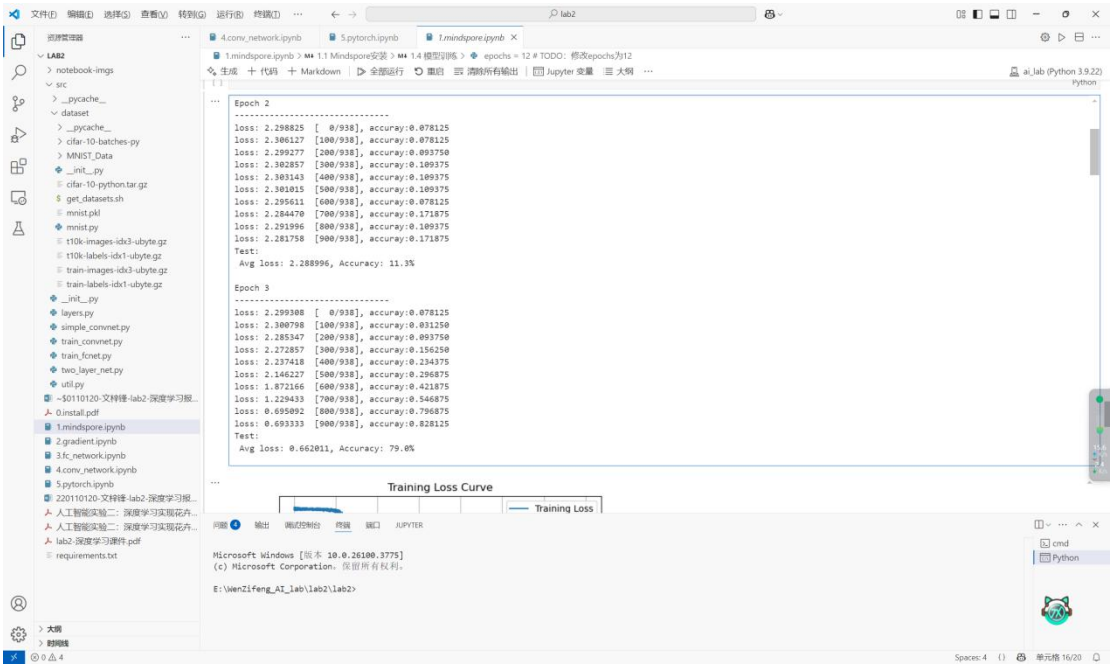
（1）简单说明做了哪些超参数调整，为什么调整这些参数

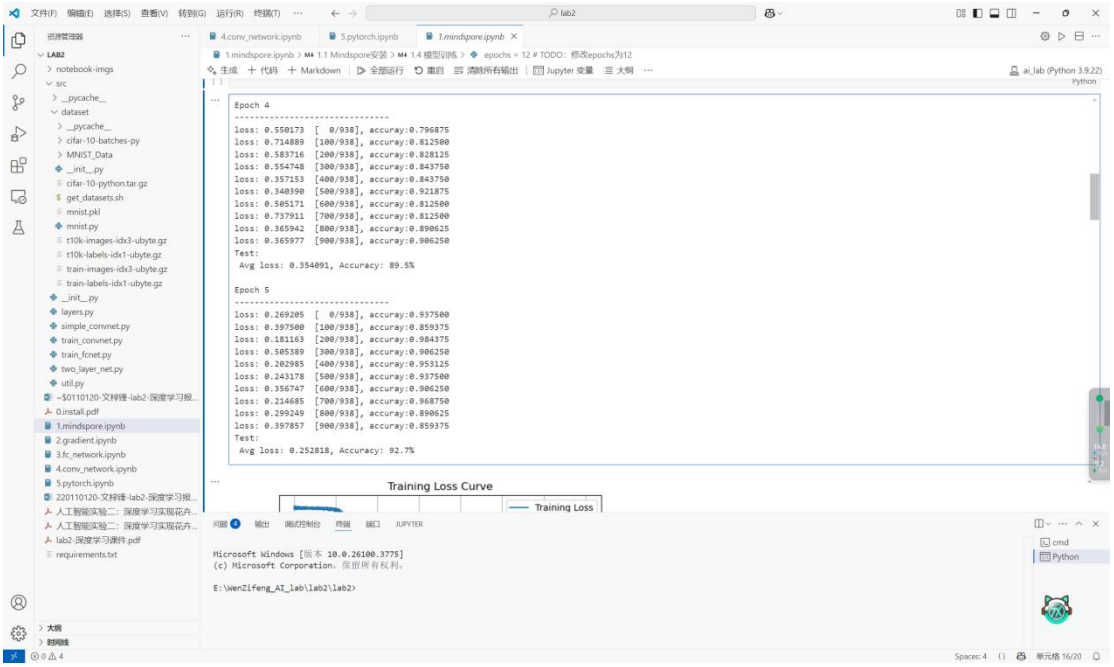
调整超参数：epoch 为 15，原来为 3

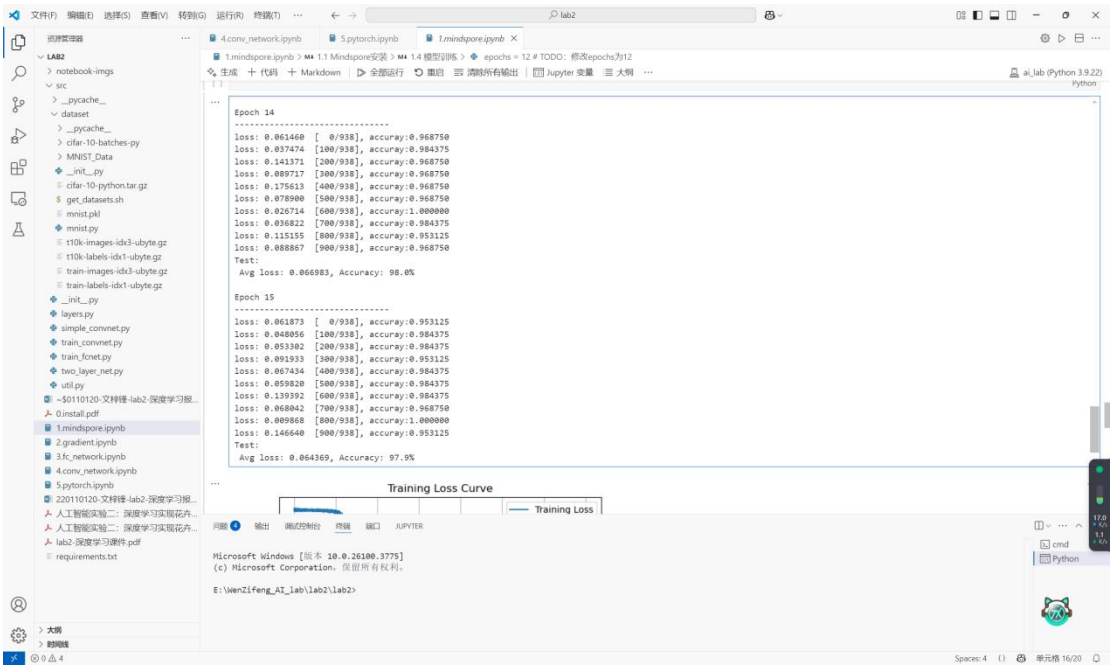
```
epochs = 15 # TODO: 修改epochs为15
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train(model, train_dataset)
    test(model, test_dataset, loss_fn)
print("Done!")

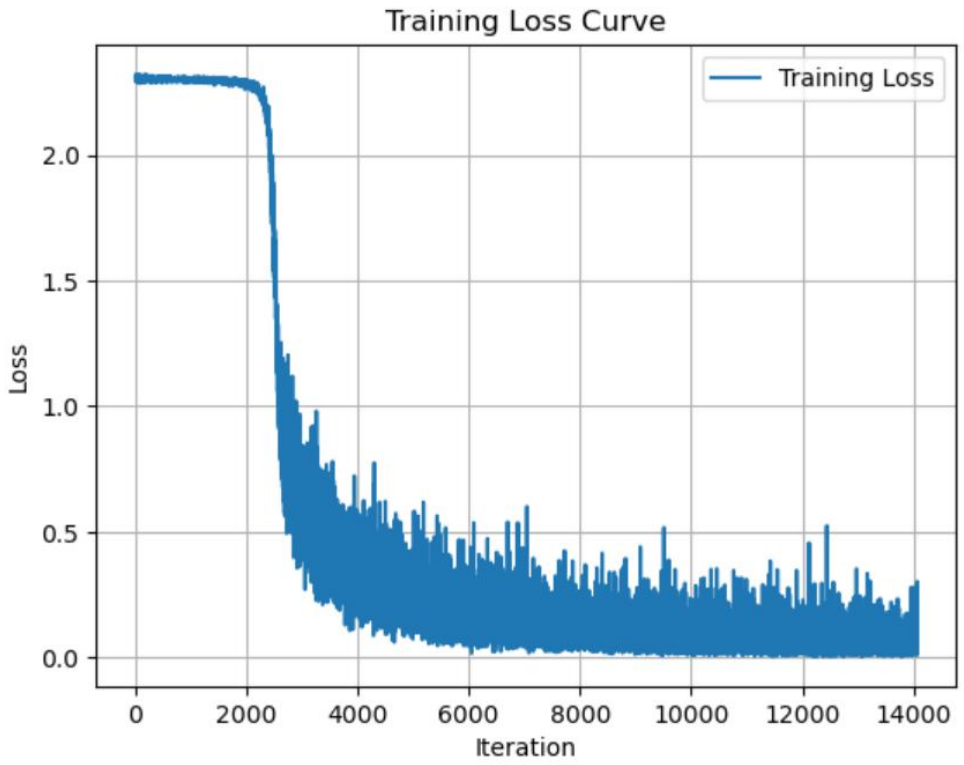
# TODO:  绘图
plt.plot(train_loss_list, label='Training Loss')
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Training Loss Curve")
plt.legend()
plt.grid(True)
plt.show()
```

使用 mindspore 进行模型训练日志截图如下图所示，在 test_dataset(mnist)数据集上能够达到 90%以上的准确率









参数调整：将 epochs 修改为 15，并且将 loss 存储下来进行绘图

可以看到在 test_dataset(mnist)数据集上能够达到 97%以上的准确率

为什么调整这些参数：

原因：在初步实验中观察到，loss 在原有轮数下尚未充分收敛，且梯度下降仍较为显著，说明模型仍在有效学习阶段。如果过早停止训练，可能导致模型性能未能达到最佳。通过适度增加训练轮数到 15，可以让模型有更多机会优化参数，进一步降低训练误差。同时，训练过程中 loss 曲线逐渐平滑下降，验证集上的准确率持续提升，最终在 test_dataset 数据集上达到 97%以上的准确率，验证了训练轮数调整的有效性。

2、反向传播和全连接网络

（1）sigmoid 函数计算图和反向传播推导过程，计算图需清楚清晰

sigmoid，sigmoid_grad，softmax 三个函数代码如下图所示：

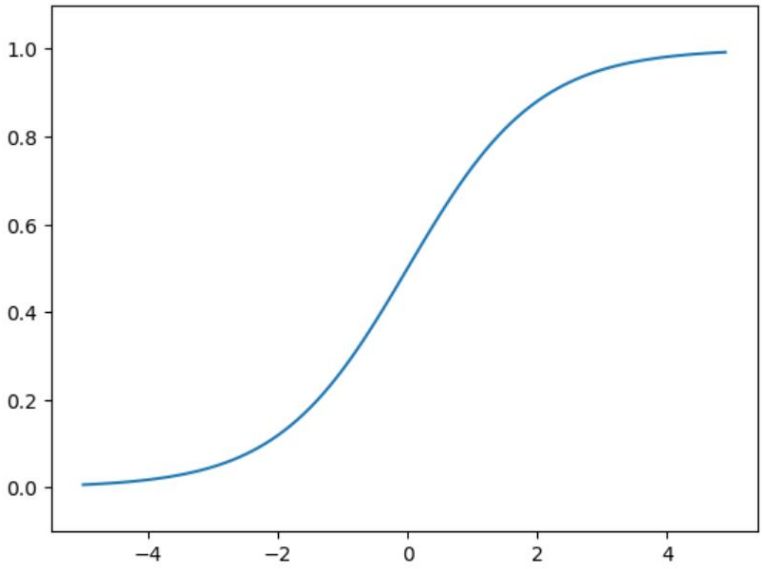
```python
def sigmoid(x):
    """
    x: sigmoid函数的输入值,numpy数组类型
    返回:sigmoid函数值,numpy数组类型
    """
    # 指数函数可直接使用numpy.exp()
    return 1 / (1 + np.exp(-x))

def sigmoid_grad(x):
    """ sigmoid函数梯度
    x: numpy数组类型
    返回: 梯度值,numpy数组类型
    """
    return sigmoid(x) * (1 - sigmoid(x))

def softmax(x):
    """
    x: numpy数组类型,需考虑多个样本即二维数组的情况
    返回: numpy数组类型
    """
    # 如果x中的值过大会导致指数计算的结果过大发生溢出：np.exp(x)会出现inf，最终的结果会出现nan，可考虑减去每一个样本对应x中的最大值
    if x.ndim == 2:
        # 批量处理，减去每一行最大值
        x = x - np.max(x, axis=1, keepdims=True)
        exp_x = np.exp(x)
        return exp_x / np.sum(exp_x, axis=1, keepdims=True)
    else:
        # 单个样本处理
        x = x - np.max(x)  # 溢出修正
        exp_x = np.exp(x)
        return exp_x / np.sum(exp_x)
```

测试结果截图如下图所示，所得到的结果在允许范围内 10^-5 以下

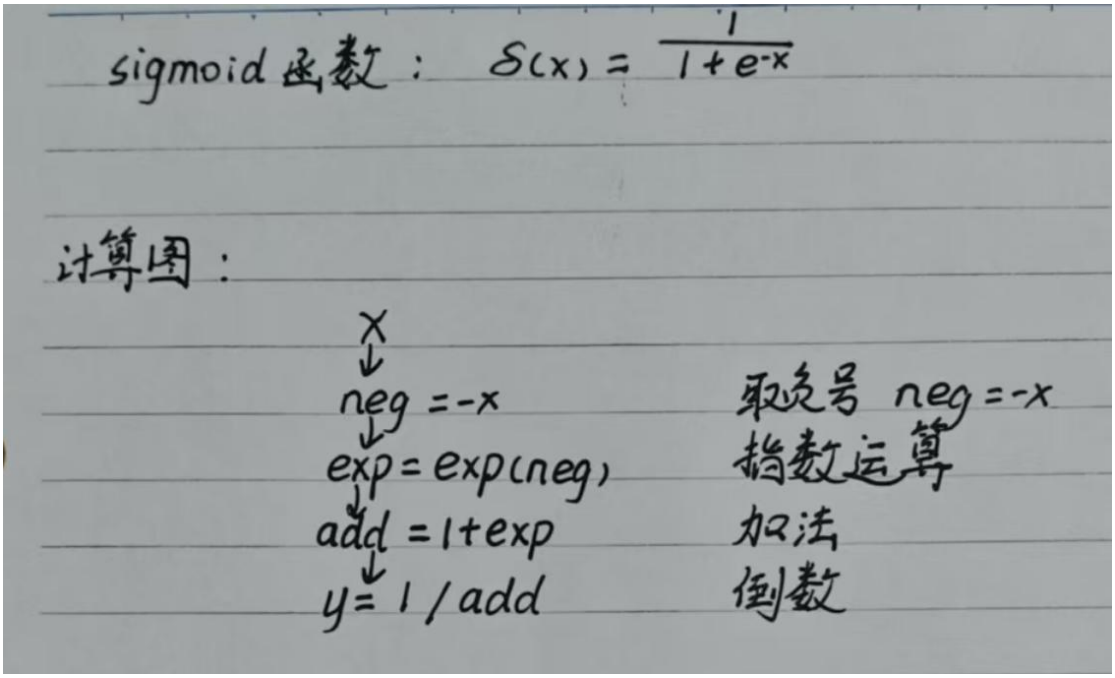
```
sigmoid check error:  [1.36999512e-09 1.36999512e-09 2.02211770e-09]
sigmoid gradient array check error: [1.01460507e-12 1.76095249e-12 5.65662794e-12]
sigmoid gradient check error: 5.893741050755352e-11
softmax check error: [3.29554753e-09 2.93507391e-09 3.76937859e-09]
softmax result sum: 1.0
batch softmax check error: [[3.29554753e-09 2.93507391e-09 3.76937859e-09]
 [4.54276600e-09 1.13362886e-09 4.32360514e-09]
 [2.38808343e-09 4.01977651e-09 1.63169295e-09]]
batch softmax result sum: [1. 1. 1.]
```



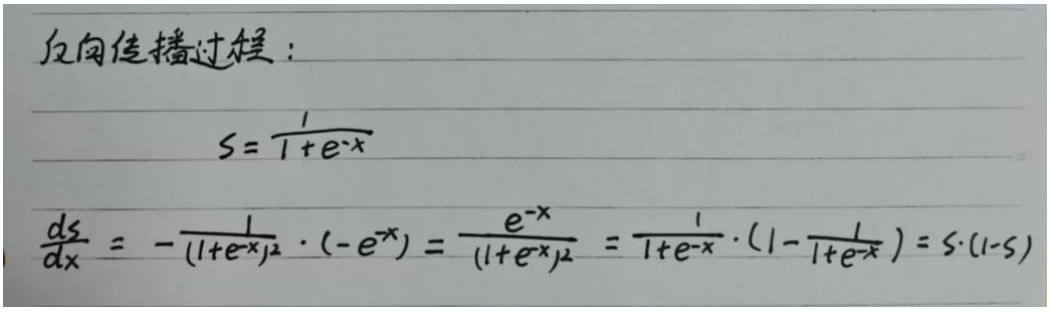
sigmoid 函数计算图和反向传播推导过程，图片可以手画拍照，需保证清楚清晰

```python
def sigmoid(x):
    """
    x: sigmoid函数的输入值,numpy数组类型
    返回:sigmoid函数值,numpy数组类型
    """
    # 指数函数可直接使用numpy.exp()
    return 1 / (1 + np.exp(-x))
```

sigmoid 函数计算图如下图所示：



反向传播推导过程如下图所示：




```python
def sigmoid_grad(x):
    """ sigmoid函数梯度
    x: numpy数组类型
    返回: 梯度值,numpy数组类型
    """
    return sigmoid(x) * (1 - sigmoid(x))
```

（2）train_fcnet.py 训练做了哪些超参数调整

```
%matplotlib inline
!cd src && python train_fcnet.py
```

✓ 21.0s

```
train size 60000
epoch 0 train acc: 0.0993, test acc: 0.1032
epoch 1 train acc: 0.7833, test acc: 0.7912
epoch 2 train acc: 0.8755, test acc: 0.8817
epoch 3 train acc: 0.8985, test acc: 0.8997
epoch 4 train acc: 0.9075, test acc: 0.9092
epoch 5 train acc: 0.9150, test acc: 0.9178
epoch 6 train acc: 0.9199, test acc: 0.9219
epoch 7 train acc: 0.9234, test acc: 0.9251
epoch 8 train acc: 0.9283, test acc: 0.9282
epoch 9 train acc: 0.9314, test acc: 0.9316
epoch 10 train acc: 0.9344, test acc: 0.9360
epoch 11 train acc: 0.9374, test acc: 0.9374
epoch 12 train acc: 0.9397, test acc: 0.9402
epoch 13 train acc: 0.9411, test acc: 0.9407
epoch 14 train acc: 0.9441, test acc: 0.9431
epoch 15 train acc: 0.9455, test acc: 0.9447
epoch 16 train acc: 0.9477, test acc: 0.9466
epoch 17 train acc: 0.9489, test acc: 0.9484
epoch 18 train acc: 0.9492, test acc: 0.9496
epoch 19 train acc: 0.9516, test acc: 0.9503
gradient compute time:  8.11091160774231 s
predict label:  [7 2 1 0] true label [7 2 1 0]
Figure(640x480)
```

将 learning_rate 调整为 0.1，将 epoch 调整为 20

```
# 读入数据
(x_train, t_train), (x_test, t_test) = load_mnist(normalize=True, one_hot_label=True)

network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)
train_size = x_train.shape[0]
batch_size = 100
learning_rate = 0.1 # TODO：修改为0.1，原来为0.01
print('train size', train_size)

train_loss_list = []
train_acc_list = []
test_acc_list = []

iter_per_epoch = max(train_size / batch_size, 1)
epoch = 20 # TODO: 修改为20，原来为5
iters_num = int(iter_per_epoch * epoch)
total_elapsed_time = 0
```

3、卷积

（1）util.py/conv_forward_naive 代码截图、train_convnet.py 代码截图，训练日志截图

conv_forward_naïve 函数代码如下图所示

```python
def conv_forward_naive(x, w, b, conv_param):
    """可以用np.lib.stride_tricks.sliding_window_view产生滑动窗口
       可以用np.pad填充
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and
    width W. We convolve each input with F different filters, where each filter
    spans all C channels and has height HH and width WW.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    During padding, 'pad' zeros should be placed symmetrically (i.e equally on both sides)
    along the height and width axes of the input. Be careful not to modfiy the original
    input x directly.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x_p, w, b, conv_param)  返回填充后的x便于反向传播直接使用
    """

    # 获取输入的形状
    N, C, H, W = x.shape
    F, _, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Pad input
    x_p = np.pad(x, ((0,), (0,), (pad,), (pad,)), mode='constant', constant_values=0)

    # Output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    out = np.zeros((N, F, H_out, W_out))

    # If b is a scalar, convert it to ndarray
    if not isinstance(b, np.ndarray):
        b = np.array([b] * F)

    # Convolution
    for i in range(N):
        for f in range(F):
            for h in range(H_out):
                for w_ in range(W_out):  # 避免与 w 冲突，这里变量改为 w_
                    h_start = h * stride
                    w_start = w_ * stride
                    x_slice = x_p[i, :, h_start:h_start+HH, w_start:w_start+WW]
                    out[i, f, h, w_] = np.sum(x_slice * w[f]) + b[f]

    cache = (x_p, w, b, conv_param)
    return out, cache
```

4.5 批处理运行结果截图如下图所示，可以看到 difference 为 2.21^10-8，满足题目需求

```
# 4维数据卷积结果校验
x_shape = (2, 3, 4, 4)
w_shape = (3, 3, 4, 4)
x = np.linspace(-0.1, 0.5, num=np.prod(x_shape)).reshape(x_shape)
w = np.linspace(-0.2, 0.3, num=np.prod(w_shape)).reshape(w_shape)
b = np.linspace(-0.1, 0.2, num=3)

conv_param = {'stride': 2, 'pad': 1}
out, _ = conv_forward_fast(x, w, b, conv_param)
correct_out = np.array([[[[-0.08759809, -0.10987781],
                          [-0.18387192, -0.2109216 ]],
                         [[ 0.21027089,  0.21661097],
                          [ 0.22847626,  0.23004637]],
                         [[ 0.50813986,  0.54309974],
                          [ 0.64082444,  0.67101435]]],
                        [[[-0.98053589, -1.03143541],
                          [-1.19128892, -1.24695841]],
                         [[ 0.69108355,  0.66880383],
                          [ 0.59480972,  0.56776003]],
                         [[ 2.36270298,  2.36904306],
                          [ 2.38090835,  2.38247847]]]])

# Compare your output to ours; difference should be around e-8
print('Testing conv_forward')
print('difference: ', rel_error(out, correct_out))
```

[4] ✓ 0.0s

```
Testing conv_forward
difference:  2.2121476575931688e-08
```

4.7 卷积反向传播计算运行结果截图如下图所示，可以看到 dx 为 1.78^10-8，dw 为 3.96^10-10，db 为 2.63^10-11，误差均小于 e-8 满足题目需求

下面是卷积反向传播实现的正确性校验，与数值微分的结果做对比，dx,dw,db的误差小于e-8。

```
# 反向传播计算结果校验
np.random.seed(231)
x = np.random.randn(4, 3, 7, 7)
w = np.random.randn(2, 3, 3, 3)
f = w.shape[0]
b = np.random.randn(f,)

conv_param = {'stride': 1, 'pad': 1}
out, cache = conv_forward_naive(x, w, b, conv_param)   #
dout = np.random.randn(*out.shape)

dx_num = eval_numerical_gradient_array(lambda x: conv_forward_naive(x, w, b, conv_param)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: conv_forward_naive(x, w, b, conv_param)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: conv_forward_naive(x, w, b, conv_param)[0], b, dout)

dx, dw, db = conv_backward_naive(dout, cache)

# Your errors should be around e-8 or less.
print('Testing conv_backward function')
print('dx error: ', rel_error(dx, dx_num))
print('dw error: ', rel_error(dw, dw_num))
print('db error: ', rel_error(db, db_num))
```

✓ 2.6s

```
Testing conv_backward function
dx error:  1.7843286351397387e-08
dw error:  3.967403416705241e-10
db error:  2.6360095355032378e-11
```

（2）train_convnet.py 代码截图，训练日志截图

代码截图如下图所示：

```python
# coding: utf-8
import sys, os
sys.path.append(os.pardir)  # 添加上层路径以引用 src 中模块
import time
import numpy as np
import matplotlib.pyplot as plt

from src.dataset.mnist import load_mnist
from src.simple_convnet import SimpleConvNet

# 读入数据
(x_train, t_train), (x_test, t_test) = load_mnist(flatten=False, normalize=True, one_hot_label=True)
print("train:", x_train.shape, "test:", x_test.shape)

# 初始化网络
network = SimpleConvNet(input_dim=(1, 28, 28),
                        conv_param={'filter_num':30, 'filter_size':5, 'pad':0, 'stride':1},
                        hidden_size=100, output_size=10, weight_init_std=0.01)

train_size = x_train.shape[0]
batch_size = 100
learning_rate = 0.1
epochs = 10
iter_per_epoch = max(train_size // batch_size, 1)
iters_num = iter_per_epoch * epochs

train_loss_list = []
train_acc_list = []
test_acc_list = []
total_elapsed_time = 0

for i in range(iters_num):
    # 随机选取 batch
    batch_mask = np.random.choice(train_size, batch_size)
    x_batch = x_train[batch_mask]
    t_batch = t_train[batch_mask]

    # 计算梯度
    start = time.time()
    grad = network.gradient(x_batch, t_batch)
    elapsed = time.time() - start

    total_elapsed_time += elapsed
    # 更新参数
    for key in ('W1', 'b1', 'W2', 'b2', 'W3', 'b3'):
        network.params[key] -= learning_rate * grad[key]

    # 记录损失
    loss = network.loss(x_batch, t_batch)
    train_loss_list.append(loss)

    # 每轮计算一次准确率
    if i % iter_per_epoch == 0:
        train_acc = network.accuracy(x_train, t_train)
        test_acc = network.accuracy(x_test, t_test)
        train_acc_list.append(train_acc)
        test_acc_list.append(test_acc)
        print("epoch {:2d} | train acc: {:.4f}, test acc: {:.4f}".format(i // iter_per_epoch, train_acc, test_acc))

print("训练总耗时: {:.2f} 秒".format(total_elapsed_time))

# 绘制 loss 曲线
plt.figure()
plt.plot(train_loss_list, label='Train Loss')
plt.title('Loss Curve')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

# 绘制 accuracy 曲线
epochs = np.arange(len(train_acc_list))
plt.figure()
plt.plot(epochs, train_acc_list, label='Train Accuracy')
plt.plot(epochs, test_acc_list, label='Test Accuracy')
plt.title('Accuracy Curve')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

训练日志截图如下图所示，test acc 的准确率为 98.69%，达到 90%以上满足题目需求：

```
1  !cd src && python train_convnet.py
✓  28m 48.6s
```

```
epoch  0 | train acc: 0.0982, test acc: 0.0979
epoch  1 | train acc: 0.9248, test acc: 0.9281
epoch  2 | train acc: 0.9594, test acc: 0.9625
epoch  3 | train acc: 0.9672, test acc: 0.9649
epoch  4 | train acc: 0.9774, test acc: 0.9749
epoch  5 | train acc: 0.9799, test acc: 0.9749
epoch  6 | train acc: 0.9841, test acc: 0.9787
epoch  7 | train acc: 0.9844, test acc: 0.9785
epoch  8 | train acc: 0.9891, test acc: 0.9840
epoch  9 | train acc: 0.9869, test acc: 0.9817
训练总耗时：783.63 秒
Figure(640x480)
Figure(640x480)
```

（2）若做选做，介绍下实现方案和实验结果

4、PyTorch

（1）自定义模型代码截图、训练日志截图，简单介绍下为什么这么设计，训练过程中做了哪些超参数调整以及为什么这么调整

使用 pytorch 自定义模型代码截图如下图所示：

```
class CustomCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)

        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(128)

        self.conv3 = nn.Conv2d(128, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)

        self.pool = nn.MaxPool2d(2, 2)
        self.dropout = nn.Dropout(0.5)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))  # 32x32 -> 16x16
        x = self.pool(F.relu(self.bn2(self.conv2(x))))  # 16x16 -> 8x8
        x = self.pool(F.relu(self.bn3(self.conv3(x))))  # 8x8 -> 4x4
        x = x.view(x.size(0), -1)  # flatten
        x = self.dropout(x)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

训练日志截图如下图所示：

```
model = CustomCNN()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

train_model(model, optimizer, epochs=9)

summary(model, input_size=(3, 32, 32))
```

✓ 2m 6.9s

```
Iteration 600, loss = 1.0333
Checking accuracy on validation set
Got 822 / 1000 correct (82.20)
```

使用 Pytorch 基于 CIFAR10 上训练神经网络，要求 10 个 epoch 内能在验证集上至少达到 70%的准确率如下图所示：在 9 个 epoch 内能在验证集上至少达到了 79%的准确率，满足题目需求

训练集上完成训练后，最后在测试集上进行测试，注意测试集只能用一次，用于模型最后的泛化能力评估。

```
best_model = model
check_accuracy(loader_test, best_model)
```

✓ 2.3s

```
Checking accuracy on test set
Got 7848 / 10000 correct (78.48)
```

（3）分别给出 5.pytorch.ipynb 中用到的两层全连接网络、三层卷积网络、自定义网络的参数量，要求有计算过程，以及使用 model.parameters()、model.state_dict()或者 pytorch-summary 等查看模型参数的输出结果截图

这里以2层的全连接网络TwoLayerFC训练为例：

1. 定义模型TwoLayerFC，只需要传入输入数据大小、hidden_layer_size、分类类别数
2. 使用SGD优化器
3. 调用train_model进行模型训练，训练一个epoch之后预计可以达到40%的准确率。

```
hidden_layer_size = 4000
learning_rate = 1e-2
model = TwoLayerFC(3 * 32 * 32, hidden_layer_size, 10)
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

train_model(model, optimizer)

summary(model, input_size=(3*32*32,))
```

[11] ✓ 7.9s

参数量计算：

第一层全连接：Linear(3072, 4000)

参数量 $= 3072 \times 4000 + 4000 = 12{,}288{,}000 + 4000 = 12{,}292{,}000$

第二层全连接：Linear(4000, 10)

参数量 $= 4000 \times 10 + 10 = 40{,}010$

总参数量：$12{,}292{,}000 + 40{,}010 = 12{,}332{,}010$

```
Epochs : 0
Iteration 0, loss = 3.1509
Checking accuracy on validation set
Got 136 / 1000 correct (13.60)

Iteration 200, loss = 1.4959
Checking accuracy on validation set
Got 391 / 1000 correct (39.10)

Iteration 400, loss = 2.5895
Checking accuracy on validation set
Got 374 / 1000 correct (37.40)

Iteration 600, loss = 1.5632
Checking accuracy on validation set
Got 432 / 1000 correct (43.20)

----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Linear-1                [-1, 4000]      12,292,000
            Linear-2                  [-1, 10]          40,010
================================================================
Total params: 12,332,010
Trainable params: 12,332,010
...
Forward/backward pass size (MB): 0.03
Params size (MB): 47.04
Estimated Total Size (MB): 47.09
----------------------------------------------------------------
```

卷积神经网络的训练跟TwoLayerFC训练过程一样，训练一个epoch之后预计可以达到45%的准确率。

```
learning_rate = 3e-3
channel_1 = 32
channel_2 = 16

model = None
optimizer = None

model = ThreeLayerConvNet(3, channel_1, channel_2, 10)
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

train_model(model, optimizer)

summary(model, input_size=(3, 32, 32))
```

✓ 8.5s

参数量计算（与 summary 一致）

Conv1（3 → 32，kernel=5x5，padding=2）：

参数 = 32 × 3 × 5 × 5 + 32 = 2,400 + 32 = 2,432

Conv2（32 → 16，kernel=3x3，padding=1）：

参数 = 16 × 32 × 3 × 3 + 16 = 4,608 + 16 = 4,624

FC（16×32×32 = 16384）→ 10：

参数 = 16384 × 10 + 10 = 163,840 + 10 = 163,850

```
Epochs : 0
Iteration 0, loss = 3.1962
Checking accuracy on validation set
Got 145 / 1000 correct (14.50)

Iteration 200, loss = 1.7173
Checking accuracy on validation set
Got 399 / 1000 correct (39.90)

Iteration 400, loss = 1.7619
Checking accuracy on validation set
Got 435 / 1000 correct (43.50)

Iteration 600, loss = 1.6900
Checking accuracy on validation set
Got 461 / 1000 correct (46.10)

----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1          [-1, 32, 32, 32]           2,432
            Conv2d-2          [-1, 16, 32, 32]           4,624
            Linear-3                  [-1, 10]         163,850
================================================================
Total params: 170,906
...
Forward/backward pass size (MB): 0.38
Params size (MB): 0.65
Estimated Total Size (MB): 1.04
----------------------------------------------------------------
```

```
model = CustomCNN()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

train_model(model, optimizer, epochs=9)

summary(model, input_size=(3, 32, 32))
```
✓ 1m 24.5s

参数量计算：

Conv1: 3->64, kernel=3x3:

Conv: 64×3×3×3 + 64 = 1792

BN: 64×2 = 128

Conv2: 64->128:

Conv: 128×64×3×3 + 128 = 73856

BN: 128×2 = 256

Conv3: 128->128:

Conv: 128×128×3×3 + 128 = 147584

BN: 128×2 = 256

FC1: 128×4×4=2048 → 256:

参数 = 2048×256 + 256 = 524,544

FC2: 256→10:

参数 = 256×10 + 10 = 2570

总参数量：

Conv 部分：1792 + 128 + 73856 + 256 + 147584 + 256 = 223,872

FC 部分：524,544 + 2570 = 527,114

总参数量：223,872 + 527,114 = 750,986

```
Iteration 600, loss = 0.7539
Checking accuracy on validation set
Got 834 / 1000 correct (83.40)

----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 64, 32, 32]           1,792
       BatchNorm2d-2           [-1, 64, 32, 32]             128
         MaxPool2d-3           [-1, 64, 16, 16]               0
            Conv2d-4          [-1, 128, 16, 16]          73,856
       BatchNorm2d-5          [-1, 128, 16, 16]             256
         MaxPool2d-6            [-1, 128, 8, 8]               0
            Conv2d-7            [-1, 128, 8, 8]         147,584
       BatchNorm2d-8            [-1, 128, 8, 8]             256
         MaxPool2d-9            [-1, 128, 4, 4]               0
          Dropout-10                 [-1, 2048]               0
           Linear-11                  [-1, 256]         524,544
           Linear-12                   [-1, 10]           2,570
================================================================
Total params: 750,986
Trainable params: 750,986
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 1.85
Params size (MB): 2.86
Estimated Total Size (MB): 4.72
----------------------------------------------------------------
```

5、实验中碰到的问题、总结、建议等

5.1 碰到的问题

先截图说明碰到的问题现象，再说明分析过程、解决方法

（一）实验中遇到的问题

1. MindSpore 环境配置问题：

初始阶段在搭建 MindSpore 框架时遇到了一些依赖冲突问题，尤其在 Windows 系统上配置 GPU 支持时较为繁琐。最终通过查阅官网教程并使用 Conda 环境隔离问题得以解决。

2. 数值微分训练速度过慢：

在使用 numerical_gradient 方法训练全连接网络时，由于计算复杂度高，训练速度明显慢于自动微分 gradient 方法，不适合用于实际训练，适合作为验证用途。

5.2 总结、建议

（一）实验总结

1. 本次实验涵盖了深度学习多个关键模块的实现与调试，包括前向传播、反向传播、梯度验证、模型训练与参数计算，全面锻炼了对神经网络底层原理的理解与动手能力。

2. 从 NumPy 手写神经网络到使用 PyTorch 框架搭建较复杂的卷积神经网络，切实体验了从底层构建到高阶调用的完整流程，有助于加深对框架设计的理解。

3. 各个模块（如 softmax、卷积等）均结合数学推导与代码实现，理论与实践结合紧密。

（二）建议与改进

1. 加入更多模型可视化和分析内容：例如在 PyTorch 部分引入 TensorBoard 或 Matplotlib 动态可视化训练过程、Loss 变化、Feature Map，有助于理解模型内部运行机制。